Closed Until Further Notice

Typing, disclosure and the engineer's timetable

Oct 26, 2026 · @Christopher Townson

Engineering is the discipline of deciding where the world is allowed to be closed. What follows tries to say why that matters, by way of two programming languages, one philosopher, and a confession.

Preface

This is an essay against a position I hold. I prefer programming languages that let me declare what things are, close the list of what may appear, and have the compiler refuse anything I did not foresee. I hold this preference while believing, as a philosopher, that it is the wrong account of how things are. The essay tries to say precisely why the preference survives the belief, and whether it should.

Two characters do most of the work. The first is Martin Heidegger, the twentieth century's great philosopher of letting things reveal themselves, who also decided in advance what everything would turn out to be, in what order, starting with the Greeks and ending with him. The second is the compiler, a clerk who refuses to proceed until the declarations are in order. No prior acquaintance with either is required; the essay will say what it needs as it goes.

The phenomenologist in what follows is me. The argument is set out as numbered propositions, each meant to be deniable on its own. Readers who find the form pompous are asked to consider that this is the point.

1. Typing as ontology

1 A type is a claim about what a thing is.

1.01 In a programming language, a type is the answer to the question "what sort of thing is this?", given in advance and checked by the compiler. An integer; a date; an account.

1.1 Every type system is therefore an ontology, whether or not its designers meant one.

1.11 An ontology is an account of what exists and what kinds of thing it may be. Philosophers have argued about this for two and a half thousand years. Engineers have argued about it for seventy, in forums, under other names.

1.2 There are two accounts of what a type is, and they are the two oldest accounts of what anything is.

1.21 On the first, the type is prior to the instance. The essence is declared; the instance participates in it; the instance can be wrong about its essence, but the essence cannot be wrong about itself. This is Plato's account.

1.22 On the second, a thing is what it shows itself as, in use. There is no essence behind the showing; the being of the thing is its involvement with other things. This is the phenomenological account, in the form Heidegger gave it.

1.221 A hammer, for Heidegger, is not a wooden handle with a metal head that also happens to be useful. It is a hammer in the hammering, and what it is for is settled by whoever is building something, not by the hammer. The analysis took several hundred pages and is widely held to be correct.

1.3 Nominal typing is Platonic. `class Account` declares an essence, and an object is an Account because it was declared one, not because of anything it does.

1.31 A sealed class is the Platonic account taken to its conclusion: a list of the only kinds of thing that may exist, with a compiler to enforce it.

```
sealed interface Posting
data class Credit(val amount: Money) : Posting
data class Debit(val amount: Money) : Posting
```

1.311 Nothing else is a Posting. Anything that claims to be one and is not on the list is refused before the program runs.

1.4 Duck typing is phenomenological. The object is an account because it answers when asked to post. Nothing was declared, and nothing needed to be.

1.5 The engineer's dispute about typing is thus not a dispute about tooling. It is the dispute about essences, conducted by people who mostly do not know that it is.

1.6 Most engineers hold both positions, in different languages, on different days. This is not inconsistency. It is the subject of this essay.

2. The most phenomenological type system in production

2 Go is the most phenomenological type system in production use, and almost nobody describes it so.

2.01 Go is a programming language designed at Google, in the main by people who had earlier designed Unix and C, for writing the servers that run large parts of the internet. It is not generally regarded as a vehicle for continental philosophy.

2.1 A Go interface is not declared by the thing that satisfies it. It is declared by the thing that needs it, usually at the point of need, usually with one or two methods.

```
type Poster interface {
    Post(p Posting) error
}
```

2.11 This says: anything that can Post is a Poster. It does not say what. It was written by whoever needed something posted, and it will admit any type that answers, including types written years later by people who have never heard of it.

2.12 `io.Reader` owns nothing. It is a description of a need, written by whoever has the need.

2.13 A `*bytes.Buffer` is a Reader when something reads from it and a Writer when something writes to it. It has no view on the matter and was never asked.

2.2 This is the hammer in the hammering, done in a type checker. The being of the type is its involvement; the involvement is conferred by whoever needs it, not by the type; and the one who needs it may never have heard of it.

2.3 The set of beings that may arrive at a Go function is therefore unbounded. Anything that answers is admitted. No permission is sought.

2.4 Go has no sum types, no sealed hierarchies and no exhaustive matching. It does not lack these by oversight. A language in which beings show themselves as they are has no use for a closed list of what they may be.

2.5 Methods in Go are functions whose first argument has been given a name. The receiver is syntax. Behaviour is not what a thing is; it is what is done with it.

2.6 Go is, in consequence, a language in which the model is never on the page. It is distributed across the functions that act on the data and the tests that constrain them. There is no single artefact at which one can point and say: that is what an account is.

2.7 The engineers who most dislike Go are, as a rule, those who most want the model on the page. This is the first thing the dislike is about. The brackets are the second.

3. The recoil

3 The phenomenologist, asked to build a system, recoils from phenomenology.

3.1 He wants the types to declare what things are. He wants the list of things that may appear to be closed. He wants the compiler to refuse any being that arrives without a ticket.

3.2 He wants, in short, a sealed hierarchy and an exhaustive match, and he wants them more the closer the system stands to money.

3.21 An exhaustive match is a piece of code that must say what it will do for every kind on the list, and will not compile until it does. It is the compiler asking “and what if it is a Debit?” and declining to be told “it won’t be”.

3.3 This looks like hypocrisy. It is better understood as a repetition.

3.4 Heidegger, the philosopher of disclosure, of letting beings be, wrote the history of Being as a sequence of epochs fixed in advance: the Greek beginning, the forgetting, the consummation in technology, the turn. Beings were free to show themselves, provided they showed themselves at the appointed station.

3.41 Gelassenheit is his word for the releasement that lets things be. It was releasement on the author’s terms. He demonstrated it across a collected edition planned at over a hundred volumes, a good many of which explain what the Greeks had meant.

3.42 Every engineer has worked with this man. He is the architect who speaks of emergence and keeps the diagram in a locked drawer.

3.5 The sealed hierarchy is a history of Being with the epochs declared in advance. Things may show themselves, but only as one of the kinds enumerated; the match is exhaustive precisely so that nothing can appear that was not foreseen.

3.6 The engineer who prefers this is not betraying phenomenology. He is doing what its founder did: wanting disclosure, on a timetable, with the stations known.

3.7 This is uncomfortable, and the discomfort is the right response. The remainder of the essay asks whether it is also the correct one.

4. The turn

4 The axis along which the dispute was drawn is the wrong axis.

4.1 Plato against phenomenology asks what a type is. It does not ask what a system may be allowed to become.

4.2 The question that actually divides the languages is whether the world is closed or open: whether the list of things that may appear is fixed, and by whom, and at what cost to reopen it.

4.21 A closed world is one in which what is not on the list does not exist. An open world is one in which the list is a courtesy.

4.3 A sealed hierarchy closes the world for one kind of thing, in one place, by one declaration, with the cost of reopening stated: a new case, a recompile, and every exhaustive match told where it is now incomplete.

4.4 A structural interface leaves the world open for one kind of need, everywhere, indefinitely, with no cost to admit a new being and no notice when one arrives.

4.5 Neither is a claim about essence. Both are claims about authority: who decides what may show up, and whether the decision is written down.

4.6 Engineering is the discipline of deciding where the world is allowed to be closed. That sentence is the job description.

4.61 The rest of the discipline is working out the consequences, mostly in meetings.

4.7 Once the axis is redrawn, the phenomenologist's preference stops being a betrayal of a position and becomes a position: that in this system, for this kind of thing, the world is closed, here, by me, until further notice.

5. Decision versus destiny

5 The closure Heidegger practised and the closure the engineer practises differ in one respect, and the respect is the whole difference.

5.1 Heidegger's closure claimed to be destiny. The epochs of Being were not a design decision with a cost attached; they were how things are, discovered rather than chosen, and the author declined to be caught choosing.

5.2 The sealed class claims to be a decision. It is declared, scoped, versioned, and reversible by recompilation. It says: closed until further notice; here is the notice period; here is what it will cost you.

5.21 Recompilation is the ten minutes after a change in which the compiler lists every place the change has broken. It is the cheapest honesty available in any profession.

5.3 What is objectionable in Heidegger's timetable, in so far as the objection is more than abuse, lies not in the closure but in the refusal to own it as a choice. A closed world presented as the structure of Being is a decision that has made itself unaccountable.

5.4 Engineering closure is honest about being a decision. It is the one thing Heidegger spent a career declining to be.

5.5 Philosophical closure that is honest about being a decision has a name. It is called method. It is no longer dogmatism, because a dogma is a decision that has forgotten it was one.

5.6 The engineer who flinches from the sealed hierarchy under Heidegger's name has therefore misnamed his instinct. The instinct is to the respectable version. What he is fleeing is a label it does not deserve.

6. The asymmetry of vices

6 The same move is a vice in philosophy and a virtue in engineering, and this is why the two cannot share an ontology.

6.1 In philosophy the cardinal error is premature closure. The dogmatist decides what may appear and then stops looking; whatever arrives outside the list is not seen, because the list is what seeing has become.

6.2 In engineering the cardinal error is the refusal to close. The service that says it manages its database connections and has never decided what a connection is; the client that handles errors inconsistently because nobody said what an error was; the model that lives in the heads of three departments who have never compared notes.

6.21 The departments are usually Finance, Operations, and whoever built the spreadsheet.

6.3 Philosophy punishes closure and engineering punishes openness. They are not two applications of one method. They are two methods, and a practitioner who confuses them will be bad at one of them.

6.4 What the two share is explication: the making explicit of a model that was only ever operative in practice. A bank runs on a tacit ontology of accounts, postings, customers and products, and at the moment it is written down three departments discover they meant different things. This is the phenomenological move, and it is why a phenomenologist can be good at the first half of the engineer's job.

6.5 What they do not share is the second half. Philosophy argues with the explicit model indefinitely. Engineering freezes it and ships.

6.6 The elision of the two, attempted over a career, fails at exactly this joint. It was always going to.

7. Phase-appropriate ontology

7 The open world and the closed world are not rival accounts of the same system. They are the same system at different times.

7.1 While one is finding out what things are, the world must be open. The vendor's documentation is wrong in places and silent about the rules the system actually enforces; the only honest posture is to let the system show itself as it is, by sending it things and recording what it does.

7.2 Once one knows, and must guarantee, the world must be closed. The types are generated, the validators written, the match made exhaustive, and the compiler enlisted to refuse what was not foreseen.

7.3 The sequence is not optional and its order is not reversible. Phenomenology first; then the timetable.

7.4 A loose, dynamic language for discovery and a strict, typed one for production is not an inconsistency of taste. It is the sequence, practised.

7.5 Heidegger's error, on this account, was not the timetable. It was writing the timetable before he had looked.

7.6 The engineer's corresponding error is the mirror image: looking indefinitely, and never writing the timetable. It has a familiar form: the project that is still discovering its domain in the third year.

8. The objection from the rhizome

8 The strongest objection to this essay does not come from Heidegger's side. It comes from a friend who reads Deleuze.

8.01 Gilles Deleuze, writing with Félix Guattari, proposed that thought has two shapes. The tree: a root, branches, a place for everything and nothing outside it. The rhizome: a tangle of connections with no root, no centre and no permission sought, which they held to be how things actually are. Trees are the instrument of states, institutions and people with org charts. The reader may sense where this is going.

8.1 The sealed hierarchy is a tree. The structural interface is as rhizomatic as a type checker gets: every node connecting to whatever it happens to connect to, no root, no permission sought.

8.2 The friend's objection runs: the preference for the tree is not an engineering judgement but a habit, acquired over two decades of being paid to want the tree, and the philosophy offered in its defence is a respectable vocabulary for a bureaucrat's instinct.

8.3 The friend is partly right, and the part that is right should be kept.

8.4 What survives is the asymmetry of vices. The rhizome is the correct account of how a domain is before it is modelled; a bank's business is a tangle, and anyone who has tried to draw it knows this. It is not an account of how a ledger may be allowed to behave at the moment of posting. At that moment the tree is not true; it is enforced, and the enforcement is the point.

8.41 A tangle that moves money is called an incident.

8.5 What also survives is the cost. Every closure is paid for in the things that cannot now appear. The engineer who closes the world without naming what he has excluded has done the bureaucrat's work after all. The one who names it has done engineering.

9. Coda

9 The preference defended here is a preference for authority over disclosure.

9.1 It is justified at the ledger, where the cost of an unforeseen being is counted in money, and dubious at the desk, where the cost of a foreseen one is counted in everything not seen.

9.2 Nothing in the argument makes it less dubious at the desk. The essay has shown that the instinct is respectable, that it is a decision rather than a destiny, and that it belongs to a phase rather than to the world. It has not shown that the phase is where one should live.

9.3 The phenomenologist who builds systems will go on wanting the timetable. He should want it with the stations named, the cost written down, and the knowledge that the train was always free to go elsewhere.

9.4 That is as far as the defence can honestly be taken. The dubiousness is left standing, because removing it would be the closure the essay is about.

9.5 Whereof one cannot speak without reflection, thereof one must write it out by hand.